



Introduction to Cloud Run functions

Welcome to the first module of this course on Cloud Run functions.

In this module, we introduce Cloud Run functions and discuss their features, and benefits.

We also review some of the use cases for Cloud Run functions, and do a lab to create your first function in the Google Cloud console.



Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04 Supported language runtimes and regions
- 05 Deploying Cloud Run functions
- 06 Lab
- 07 Review

In this module, you learn about Cloud Run functions, their features and benefits, and use cases.

We'll discuss the supported language runtimes and regions for deploying Cloud Run functions, and how they're built and deployed on Google Cloud.

You'll also complete a lab to develop and deploy Cloud Run functions from the Google Cloud console and the gcloud CLI.

Finally, we review what was discussed in this module.



Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04 Supported language runtimes and regions
- 05 Deploying Cloud Run functions
- 06 Lab
- 07 Review

Let's start by first discussing what are Cloud Run functions.

What are Cloud Run functions?



- Fully managed serverless execution environment
- Scalable functions-as-a-service (FaaS) platform
- No need to provision infrastructure or manage servers
- Can be used to connect cloud services
- Integrated with Google Cloud Observability for logging and monitoring
- Two product versions:
 - Cloud Run functions
 - Cloud Run functions (1st gen)

Cloud Run functions is a fully managed serverless execution environment on Google Cloud for you to build, deploy, and run functions.

It is a scalable functions-as-a-service platform, where there is no need to provision any infrastructure or manage individual servers.

Depending on your requirements, you can connect to other cloud services from Cloud Run functions.

It's fully integrated with Google Cloud Observability for observability and diagnosis.

There are two versions of Cloud Run functions:

- Cloud Run functions, formerly known as Cloud Functions (2nd gen), which deploys your function as services on Cloud Run, allowing you to trigger them using Eventarc and Pub/Sub.
- Cloud Run functions (1st gen), formerly known as Cloud Functions (1st gen), the original version of functions with limited event triggers and configurability.

See [Cloud Run functions version comparison](#) for more information.

What is Cloud Run?

- A managed compute platform
- Runs containers on Google's infrastructure
- Supports source-based deployment that builds containers for you
- Build full-featured applications with other Google Cloud services



Cloud Run is a managed compute platform that lets you run containers directly on top of Google's scalable infrastructure.

You can deploy code written in any programming language on Cloud Run if you can build a container image from it.

You can use the [source-based deployment](#) option that builds the container for you, when developing your application in Go, Node.js, Python, Java, .NET Core, or Ruby.

Cloud Run works well with other services on Google Cloud, so you can build full-featured applications without spending too much time operating, configuring, and scaling your Cloud Run service.

A Cloud function:

- Is a simple piece of code that implements a single-purpose function.
- Can be invoked directly from an HTTP request.
- Can be triggered in response to an event generated from cloud infrastructure and services.
- Executes in a fully managed environment.
- Is written in one of many supported programming languages:
 - Node.js, Python, Go, Java, .NET, Ruby, and PHP



A Cloud function is simple code that you write that serves a single piece of functionality.

It's triggered by an event that is generated by cloud infrastructure and other services.

Cloud functions execute in a fully managed serverless environment.

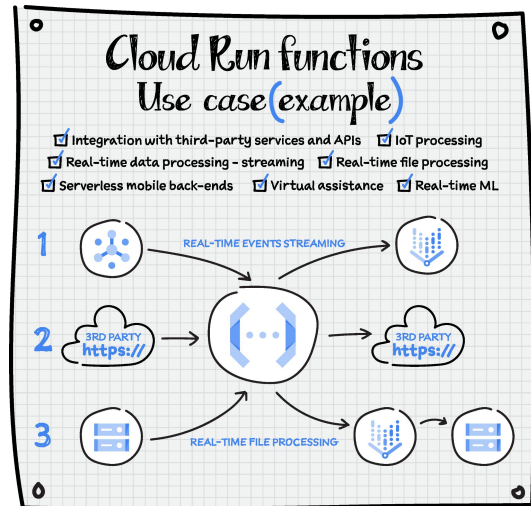
You can develop Cloud functions in any [supported language](#).

Slide graphic from Sketch notes:

<https://cloud.google.com/blog/topics/developers-practitioners/learn-cloud-functions-snap>

Using Cloud Run functions

- A connective layer to connect and extend cloud services.
- Examples:
 - Respond to a file upload to Cloud Storage
 - Respond to a message published to a Pub/Sub topic



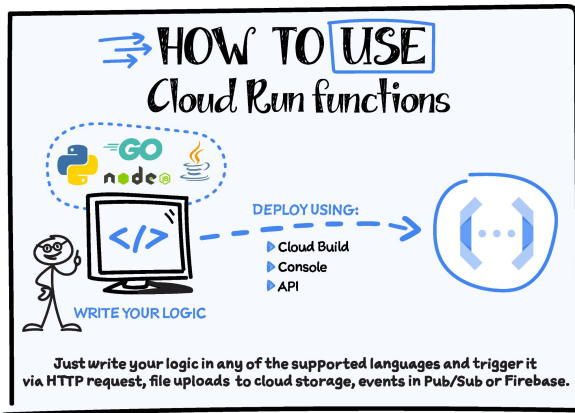
Cloud Run functions provides a connective layer of logic that lets you write code to connect and extend cloud services.

For example, a Cloud function can respond to a file upload to Cloud Storage, or an incoming message on a Pub/Sub topic.

Slide graphic from Sketch notes:

<https://cloud.google.com/blog/topics/developers-practitioners/learn-cloud-functions-snap>

To use Cloud Run Functions:



- Write your function logic in any of the supported programming languages.
- Deploy the function.
- Trigger the function from:
 - An HTTP request
 - An Event source

To set up and use Cloud Run functions, you:

- Develop the function code or logic in one of the supported programming languages.
- Deploy the function. You can deploy the function using the Google Cloud Console or the gcloud CLI.
- Set up a trigger for the function to execute in response to HTTP requests or supported cloud events.

Slide graphic from Sketch notes:

<https://cloud.google.com/blog/topics/developers-practitioners/learn-cloud-functions-snaps>



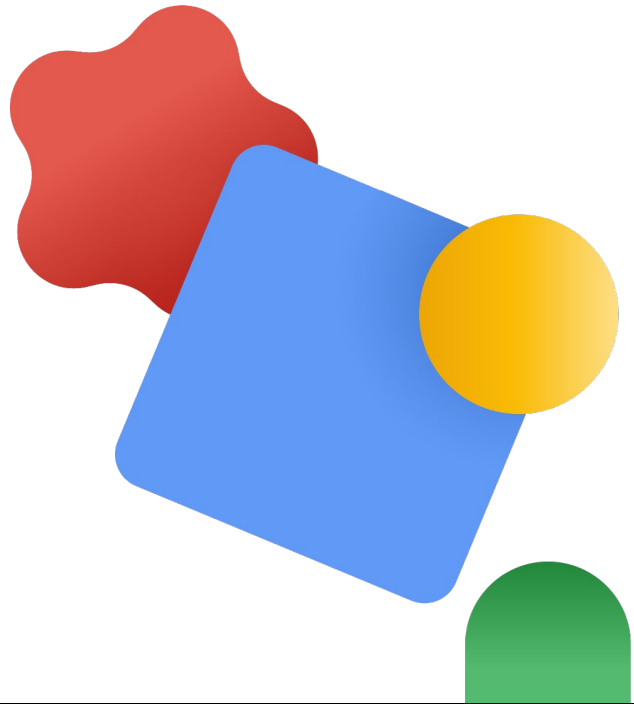
Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04 Supported language runtimes and regions
- 05 Deploying Cloud Run functions
- 06 Lab
- 07 Review

Let's now discuss the features and benefits of Cloud Run functions, and the use cases for deploying them.

Features and Benefits



Here are some of the features and benefits of using Cloud Run functions:

Cloud Run functions

Features:

- ✓ Can be locally developed and tested
- ✓ Seamless authentication
- ✓ HTTP and event-driven
- ✓ Integrates with cloud databases
- ✓ Portable using open source frameworks

Benefits:

- ✓ Augments existing cloud services
- ✓ Serverless
- ✓ Auto scalable
- ✓ Observable
- ✓ Pay as you go

Features of Cloud Run functions include:

- Local development and testing: You can develop and test functions in a local development environment before deploying to Cloud Run functions.
- Seamlessly authenticate your Cloud functions with other Google Cloud services using service accounts.
- HTTP and event-driven: Cloud Run functions execute in response to cloud events and HTTP requests.
- Integration with Cloud SQL, Bigtable, Spanner, and Firestore databases.
- Portability: A function can run in any standard runtime environment for one of the supported languages.

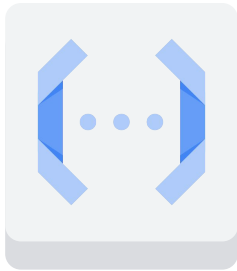
Benefits of Cloud Run functions include:

- Augmenting and extending existing cloud services with programming logic.
- Serverless. The software and infrastructure are fully managed by Google. You don't need to manage servers, update frameworks, or patch operating systems.
- Autoscaling. Resources are automatically provisioned in response to events. A function can scale from a few invocations a day to many millions of invocations without requiring any additional work.
- Observable. Cloud Run functions are integrated with Google Cloud Observability monitoring and logging tools for observability and error diagnosis.
- Pricing. A pay as you go pricing model is used. The cost is based on the

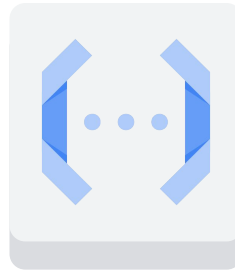
- number of function invocations, how long the function runs (compute time), and any data transfer fees for outbound network traffic.

Types of Cloud Run functions

HTTP functions



Event-driven functions

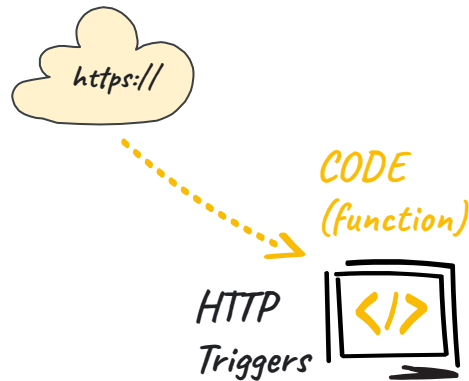


There are two types of Cloud Run functions:

- **HTTP functions**, which handle HTTP requests, and
- **Event-driven functions**, which handle events from your cloud environment.

HTTP function

- Used to handle HTTP requests.
- Provides a URL endpoint.
- Uses HTTP triggers.
- Ideal for webhooks or single use endpoints.
- Supports authenticated or unauthenticated requests.



Use HTTP functions with HTTP triggers when you want to invoke a function with an HTTP(S) request.

When you specify an HTTP trigger for a function, the function is assigned a URL at which it can receive requests.

An HTTP trigger enables a function to run in response to HTTP(S) requests.

A good use case for using HTTP functions is to implement a webhook or API that handles http requests from other services.

By default, requests to HTTP functions require authentication. You can choose to allow unauthenticated requests during function deployment.

We discuss securing Cloud Run functions in more detail in a later module of this course.

Implementing HTTP functions

HTTP functions

Write a function in your preferred language which:

- Accepts request/response arguments
- Processes the request
- Sends an HTTP response

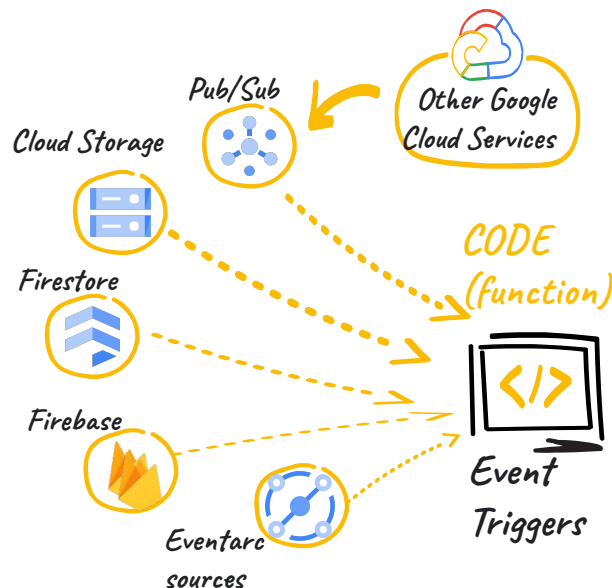
To implement an HTTP function, you write an HTTP handler function and register it with the functions framework for your programming language.

The handler accepts the request and response arguments, processes the request based on the request method, and sends back an HTTP response.

If the function creates any background tasks like threads, processes, or Promise objects, they must be terminated before the response is sent.

Event-driven function

- Responds to cloud events
- Is implemented using:
 - CloudEvent functions
 - Background functions
- Uses event triggers for:
 - Pub/Sub
 - Cloud Storage
 - Firestore
 - Firebase
 - Eventarc sources
- Integrates with other Google Cloud services that support Pub/Sub



Use event-driven functions when you want a function to be invoked automatically in response to an event that occurs in your cloud environment.

You implement event-driven functions using CloudEvent functions or Background functions depending on your chosen language runtime, and if you're using Cloud Run functions or Cloud Run functions (1st gen).

Event-driven functions use event triggers which react to events within your Google Cloud project. Event triggers are supported by many Google Cloud services based on if the cloud function is from Cloud Run functions or Cloud Run functions (1st gen).

You can use event triggers for:

- Pub/Sub
- Cloud Storage
- Firestore
- Firebase
- and Eventarc sources.

You can also integrate Cloud Run functions with any other Google Cloud service that supports Pub/Sub as an event bus.

We discuss triggers in more detail in a later module of this course.

Implementing event-driven functions

CloudEvent functions

- Based on the CloudEvents specification.
- Registered with the Functions Framework.
- Event data is in CloudEvents format.
- Used by Cloud Run functions (1st gen) [some runtimes] and Cloud Run functions.

Background functions

- Older style event-driven function.
- Event data is based on event type.
- Used by Cloud Run functions (1st gen).

Event-driven functions are executed in response to an event like receipt of a new Pub/Sub message or deletion of a file in Cloud Storage.

You implement an event-driven function as a CloudEvent function or a Background function.

CloudEvent functions are:

- Based on the [CloudEvents](#) industry standard specification for describing event data.
- Registered with the [Functions Framework](#), an open source library that wraps user functions within a persistent HTTP application.
- Used by Cloud Run functions with all language runtimes.
- Used by Cloud Run functions (1st gen) with .NET, Ruby, and PHP.

Background functions are:

- Older style event-driven functions that receive event data based on the type of event.
- Used by Cloud Run functions (1st gen) with Node.js, Python, Go, and Java runtimes.

Features of Cloud Run functions

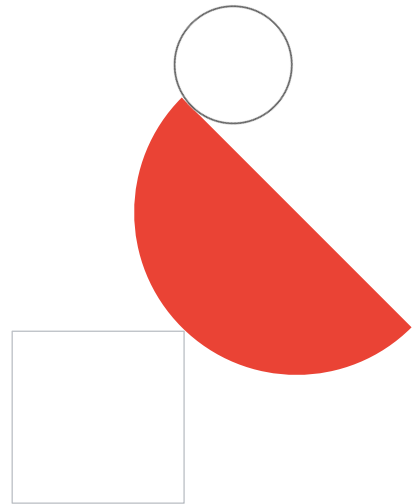
- ✓ Larger instances
- ✓ Increased concurrency
- ✓ Longer request processing
- ✓ Fast rollbacks and traffic splitting
- ✓ Extensibility and portability
- ✓ Many more event sources
- ✓ Standardized event schema
- ✓ Improved developer experience

Cloud Run functions have the following features:

- Function instances with up to 32 GiB of RAM and 4 vCPUs.
- Process up to 1000 concurrent requests with a single function instance. This capability reduces cold starts and improves overall latency when scaling.
- Run time limit of up to 60 minutes for HTTP functions, and up to 10 minutes for event-driven functions to process longer request workloads.
- With a new revision created every time you deploy your function, you can split traffic between different revisions or rollback your function to a prior revision.
- Leverage Cloud Run's scalable container platform to move your function to Cloud Run or to Kubernetes.
- With Eventarc, event-driven functions can be triggered from more than 90 Google, external SaaS event sources, and custom sources by publishing to Pub/Sub.
- With the industry-standard CloudEvents format, you can simplify your event handling code.
- In Google Cloud console configure your function's triggers, track deployments, test your functions, and view function metrics.

View the [documentation](#) on the comparison between Cloud Run functions and Cloud Run functions (1st gen).

Use cases



Some of the use cases for Cloud Run functions include:

Use cases



Data Processing

Respond to Cloud Storage events when a file is created, updated, or deleted.

Validate and transform data, process images, and transcode videos.

API Webhooks/APIs

Process webhook and API calls originating from external systems that can send HTTP requests.



Mobile device backend

Develop a mobile device backend responding to events triggered by Firebase, Google's mobile platform for app developers.



IoT

Stream device data into Pub/Sub, and launch Cloud Run functions to process, transform, and store data.

- **Data processing**
By responding to Cloud Storage events on files and objects, you can validate and transform data, process images and videos for further downstream processing.
- **Webhooks and APIs**
With HTTP functions, you can support webhook and API calls that originate from external systems by processing HTTP requests.
- **Mobile device backend** that responds to events triggered by [Firebase](#).
With Cloud Run functions, you can develop mobile device backend services that responds to events triggered by Firebase and Firestore.
- **IoT**
You can develop IoT (internet-of-things) applications with Cloud Run functions by processing and storing data from devices streamed into Pub/Sub.



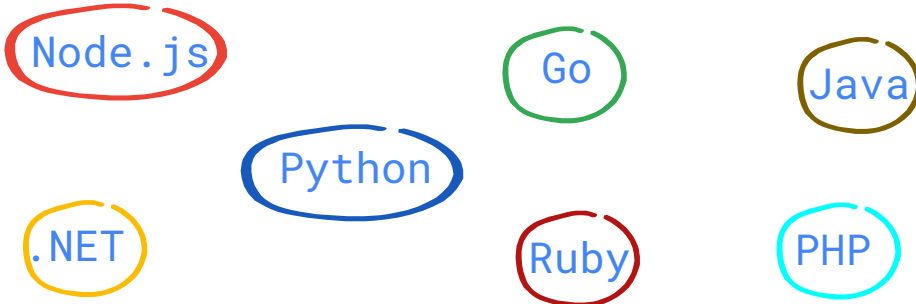
Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04** Supported language runtimes and regions
- 05 Deploying Cloud Run functions
- 06 Lab
- 07 Review

In this section, we discuss the language runtimes and regions supported by Cloud Run functions.

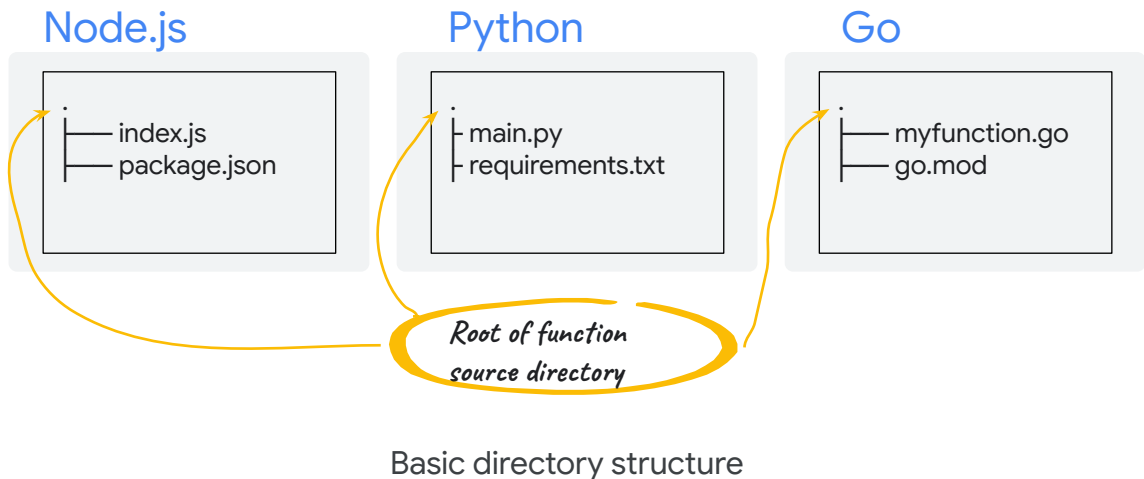
Language runtimes



You can develop Cloud Run functions in any of these supported languages.

There are specific versions of each language runtime that is supported. Please refer to the [Cloud Run functions documentation](#) for the version numbers.

Source directory structure



The language runtime that you choose and the type of function that you want to write determine the structure of your code and function implementation. For Cloud Run functions to locate your function definition, each language runtime has specific requirements for structuring your source code.

Here are the basic directory structures for some of the languages.

For Node.js, by default, Cloud Run functions loads source code from a file named `index.js` at the root of your function directory.

To specify a different main source file, use the `main` field in your `package.json` file. The `package.json` file also includes the [Functions Framework for Node.js](#) as a dependency, along with any additional library dependencies needed by your function.

For Python, Cloud Run functions loads source code from a file named `main.py` at the root of your function directory. The `requirements.txt` file also includes the [Functions Framework for Python](#) as a dependency and any additional library dependencies needed by your function.

For Go, your function must be in a Go package at the root of your project. The `go.mod` file includes the [Functions Framework for Go](#) as a dependency, and any additional library dependencies needed by your function.

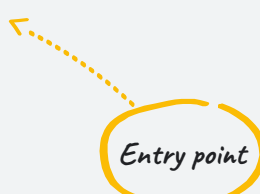
For more details on creating your source code directory structure for your preferred language, refer to the [Cloud Run functions documentation](#).

Function entry point

- Code that runs when a function is invoked
- Must be defined in your source code in the *main* file or root package
- Can be a function or class depending on programming language
- Specified when the function is deployed

Node.js

```
const functions =  
require('@google-cloud/functions-framework');  
  
// Register an HTTP function with the  
Functions Framework  
functions.http('myHttpFunction', (req, res)  
=> {  
  // Your code here  
  
  // Send an HTTP response  
  res.send('OK');  
});
```



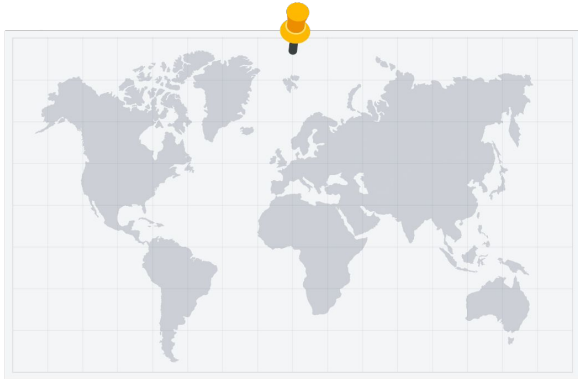
The function entry point is the code that is executed when the Cloud Run function is invoked. You specify this entry point when you deploy your function.

Your source code must define an entry point for your function and must be defined in your main file or root package.

Depending on the programming language, the entry point is a function or a class.

You specify this entry point when you deploy your function.

Cloud Run functions locations



- ✓ Regional
- ✓ Managed by Google
- ✓ Redundantly available across all zones in a region
- Regional availability:
 - Depends on Cloud Run functions generation
 - Affects tier pricing

The infrastructure that runs a Cloud Run functions is located in a specific region and is managed by Google to be redundantly available across all the zones within that region.

When selecting a region to run your Cloud Run functions in, your primary considerations should be latency and availability.

You can generally select the region closest to your Cloud Run function's users, but you should also consider the location of the [other Google Cloud products and services](#) that your app uses. Using services across multiple locations can affect your app's latency, and [pricing](#).

Cloud Run functions and Cloud Run functions (1st gen) have different regional availability.

<https://cloud.google.com/functions/docs/locations>



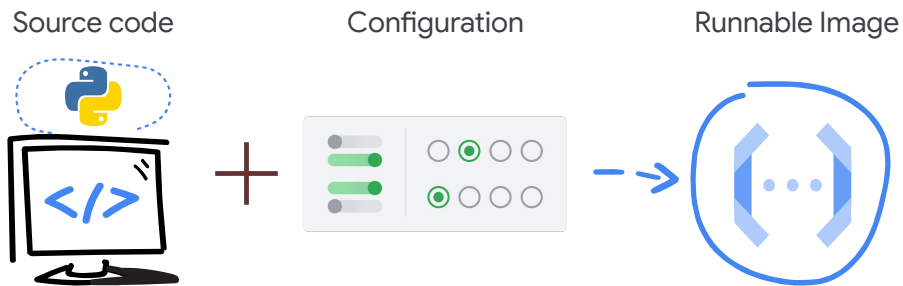
Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04 Supported language runtimes and regions
- 05** Deploying Cloud Run functions
- 06 Lab
- 07 Review

Let's review the process to build and deploy Cloud Run functions from source code.

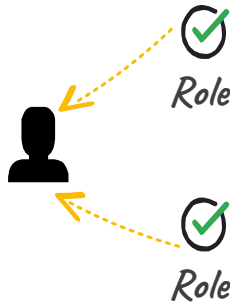
Deployment basics



The deployment process takes your function source code and configuration settings, and builds a runnable image.

Cloud Run functions automatically manages the image in order to handle requests to your function.

Required user roles



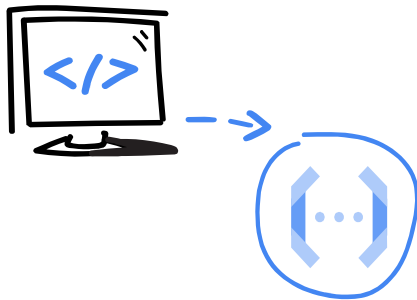
To be able to deploy Cloud Run functions, a user must have these IAM roles:

- Cloud Functions Developer (or a role with the same permissions)
- Service Account User

A user deploying Cloud Run functions must have the Cloud Functions Developer IAM role or a role that includes the same permissions.

To deploy Cloud Run functions, a user must also be assigned the Service Account User IAM role on the Cloud Run functions runtime service account.

Deployment process



Deploy your function:

- From the Google Cloud console
- Using Cloud Build
- Using Cloud Code
- With the gcloud CLI

gcloud CLI

```
gcloud functions deploy YOUR_FUNCTION_NAME \
[--gen2] \
--region=YOUR_REGION \
--runtime=YOUR_RUNTIME \
--source=YOUR_SOURCE_LOCATION \
--entry-point=YOUR_CODE_ENTRYPOINT \
TRIGGER_FLAGS
```

You can deploy your function from the Google Cloud console, using Cloud Build, Cloud Code, or with the gcloud CLI. To deploy Cloud Run functions, specify the gen2 flag.

The region flag specifies the region in which to deploy your function. The runtime flag specifies the language runtime of your function.

The source flag specifies the location of your function source code. The function can be located on your local machine, in Cloud Storage, or in a source repository.

Specify a function or class name in your source code as the entry point of your cloud function with the entry-point flag. The function or class code is executed when your function runs.

Cloud Code is a tool that simplifies the development and deployment of Cloud Run functions. It allows you to create, deploy, and invoke functions directly within your IDE, making the process more efficient.

To specify the type of trigger and related configuration for your function, provide trigger flags.

Refer to the gcloud functions deploy command [documentation](#) for more details.

Deployment sources

You can deploy your function from a:

- Local machine

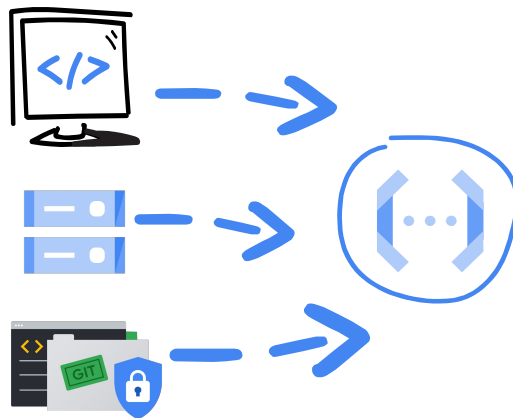
`--source=/path/root_directory_of_function_source_code`

- Cloud Storage bucket

`--source=gs://bucket/folder/function.zip`

- Source repository (1st gen only)

`--source=https://source.developers.google.com/projects/PROJECT_ID/repos/REPOSITORY_NAME`



You can specify the location of your function source code for deployment with the source flag. You can deploy your function from:

- **A local machine**

- The value of the source flag is a local file system path to the root directory of the function source code.
- Optionally, use the stage-bucket flag to specify a Cloud Storage bucket to upload your source code to as part of deployment.
- You can exclude unnecessary files with the .gcloudignore file.

- **Cloud Storage**

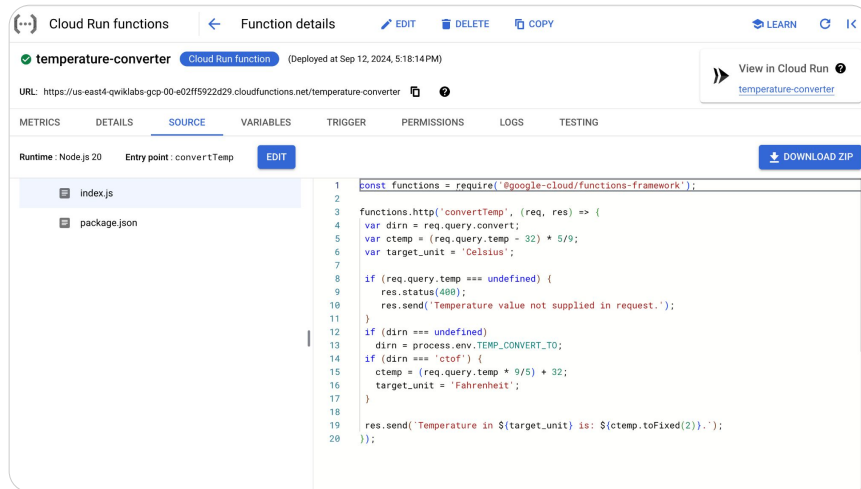
- The value of the source flag is the cloud storage path to the bucket that contains the function source code packaged as a zip file.
- Your function source files must be located at the root of the zip file.
- In Cloud Run functions (1st gen), the account performing the deployment needs permission to read from the bucket.
- In Cloud Run functions, the Cloud Run functions service agent needs permission to read from the bucket.

- **A source repository**

- The value of the source flag is a source repository reference to the location of your function source code.
- You can deploy your function source code from a revision in your repository by specifying revisions/revision name in the source repository path. To specify the location of the source code other than the repository root, append the paths/source_directory_path to the

- source repository path.
- The Cloud Run functions service agent must have the Source Repository Reader (roles/source.reader) IAM role on the repository.
- Deploying from Cloud Source Repositories also enables you to deploy code hosted in a GitHub or Bitbucket repository.

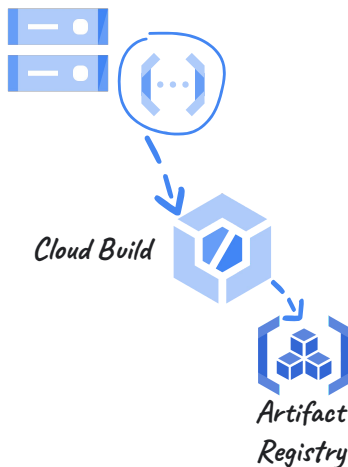
Deployment sources



You can also write and deploy a function directly from the Google Cloud console using the provided inline editor.

The editor contains a left pane to view and select source files and the right pane to edit a source file.

Build process



Cloud Run functions:

- Stores your function code in Cloud Storage when deployed.

Cloud Build:

- Automatically builds your code into a container image.
- Pushes the image to Artifact Registry.

Build Process:

- Executes in your project.
- The Cloud Build API must be enabled.
- Provides access to all build logs through Cloud Logging.

Let's review the build process for Cloud Run functions.

When you deploy your function's source code to Cloud Run functions, that source is stored in a Cloud Storage bucket.

Cloud Build is a service that executes builds on Google Cloud infrastructure.

Cloud Build automatically builds your function source code into a container image, and pushes that image to Artifact Registry.

Cloud Run functions then accesses this image when it needs to run the container to execute your function.

The process of building the image is entirely automatic and requires no direct input from you.

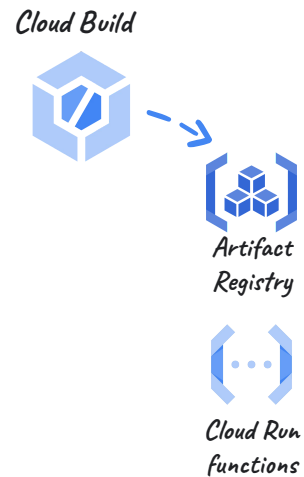
The Cloud Build API must be enabled for your project.

All the resources used in the build process execute in your own user project, and you have access to all the build logs through Cloud Logging.

Image registry

Artifact Registry:

- A Google Cloud service used to store and manage software artifacts like container images.
- Integrates with Cloud Build.
- Used by Cloud Run functions to store function images.



Cloud Run functions uses Artifact Registry to store the images built from your function source code.

Artifact Registry is a service that is used to store and manage software artifacts in private repositories, including container images, and language packages.

Artifact Registry integrates with Cloud Build to store the packages from your builds.



Agenda



- 01 Cloud Run functions
- 02 Features and benefits of Cloud Run functions
- 03 Cloud Run functions use cases
- 04 Supported language runtimes and regions
- 05 Deploying Cloud Run functions
- 06 Lab**
- 07 Review

Let's now do a lab to develop and deploy Cloud Run functions.

Lab



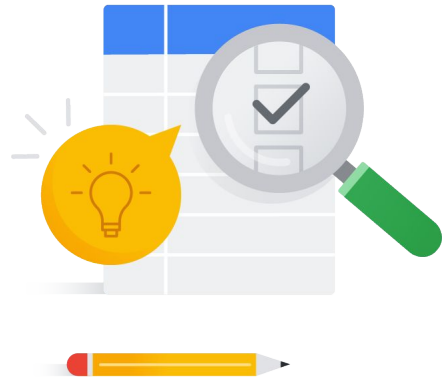
45 min



Individual

Develop and Deploy Cloud Run Functions

Create simple functions with the Google Cloud console and the gcloud CLI.



In this lab, you create and deploy simple Cloud Run functions with the Google Cloud console and the gcloud CLI.

Lab Instructions



45 min



Individual



Tasks

- Write a function that responds to HTTP requests, and deploy the function from the Google Cloud console.
- Write a function that responds to Cloud Storage events, and deploy the function using the gcloud CLI.
- Write unit tests for your function and test your function before deployment.
- Deploy multiple revisions of a function and split traffic between revisions.

1

Write an HTTP function.

2

Write an event-driven function for Cloud Storage events.

3

Write unit tests to test your function locally.

4

Deploy multiple revisions of a function.

You'll first write an HTTP function to process and respond to HTTP requests, and deploy the function in Google Cloud Console to Cloud Run functions.

Next, you'll write an event-driven function to respond to Cloud Storage events, when a file is copied to a Cloud Storage bucket. You'll deploy this function using the gcloud CLI.

When developing functions, it's a good practice to develop unit tests for your function before deploying them. In this task in the lab, you write unit tests for your function and test them locally before deploying the function.

Cloud Run functions support multiple deployed revisions of a function. In this task in the lab, you deploy more than one revision of a function and split traffic between the two revisions.



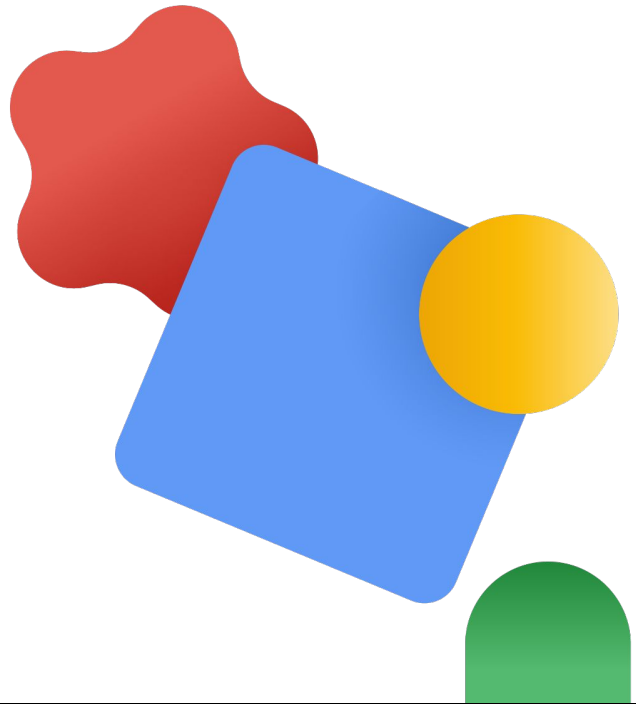
Agenda



- | | |
|----|--|
| 01 | Cloud Run functions |
| 02 | Features and benefits of Cloud Run functions |
| 03 | Cloud Run functions use cases |
| 04 | Supported language runtimes and regions |
| 05 | Deploying Cloud Run functions |
| 06 | Lab |
| 07 | Review |

Let's review what was discussed in this module.

Review: Introduction to Cloud Run functions



In this module, we introduced Cloud Run functions and discussed the types of Cloud Run functions, their features and their benefits.

We reviewed some of the use cases for Cloud Run functions, and completed a lab to create functions in the Google Cloud console, and with the gcloud CLI.

In this module, you learned about:

01	Cloud Run functions
02	Features and benefits of Cloud Run functions
03	Cloud Run functions use cases
04	Supported language runtimes
05	Deploying Cloud Run functions



You learned that Cloud Run functions is a fully managed, serverless, functions-as-a-service platform that enables you to create single-purpose functions that can be deployed and run on Google Cloud.

You learned about the features and benefits of Cloud Run functions, that include local development and testing, integration with Cloud databases, and auto-scalability, and about the some of their use cases, that include webhooks and APIs, and data processing.

We reviewed the language runtimes that are supported by Cloud Run functions, and their basic directory structure requirements when developing in Node.js, Python, and Go.

You learned how Cloud Run functions are built and deployed, and completed a lab on deploying Cloud Run functions from the Google Cloud console and the gcloud CLI.